

inc-mb-fidelity-inc____fidelity-kary-f-k4-inc

An AgentMPI run analysis

Abstract

This is the paper’s own contribution measured in the one configuration where the reduction actually loses information, and it is the best cell in the campaign on every cost axis and indistinguishable from the worst on quality. A k -ary tree with $k = 4$ over eight ranks folds four reports in one variadic application, so the whole reduction is three applications at fold depth 2 rather than seven at depth 3 or 7. The measured counts are exactly the closed forms: **fold_depth**: $2 = \lceil \log_4 8 \rceil$, three agent calls $= \lceil (p - 1)/(k - 1) \rceil = \lceil 7/3 \rceil$, seven messages $= p - 1$. It completes in 51.77 s against the binomial tree’s 80.27 s and the chain’s 634.43 s, emits 1,323 output tokens against 2,492 and 2,623, and costs \$0.0275 against \$0.0500 and \$0.0524. And it retains exactly the same 37 of 96 items — the same identifiers, ranks 0, 1 and 2 complete plus F-3-0, ranks 4 through 7 erased entirely. That is the honest form of the k -ary argument and it is not the form the paper states. The published justification is that a lossy operator’s quality decays in fold depth, so halving the depth should buy fidelity; this campaign refutes that premise, because depth 2 and depth 7 give byte-identical results. What survives, and what these cells establish, is stronger for being simpler: since an agent has no port count, a wide tree performs fewer operator applications for the same collective, and under a uniform enforced budget that is pure saving. Collective selection here is a cost decision, and k -ary wins it outright.

1 What this run is

property	value
run	inc-mb-fidelity-inc__fidelity-kary-f-k4-inc
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	broker $\times$ 8, worker $\times$ 10
events	63
wall time	51.77 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.41×	total busy time over wall time
parallel efficiency	17.7%	achieved parallelism per rank
idle fraction	82.3%	rank-seconds paid for and unused
peak ranks busy	2	of 8 in the world
serial fraction of active time	50.3%	time with exactly one rank busy
load imbalance	4.77×	slowest rank’s busy time over the mean
coordination share	20.0%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	10	fresh agent processes that took over a rank role
agent calls	3	invocations that reached a model
agent latency p50 / p95	28.01 / 30.71 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	1,537	0 deferred under rendezvous
tokens in / out	2,560 / 1,323	prompt and completion
cost	\$0.0275	0.0208 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 4s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[note]** Parallel efficiency is 17.7%: 82.3% of the rank-seconds paid for went unused.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -221.63\text{s}$), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.519$. Any latency prediction from this run’s calibration is a median, not a model.
- **[note]** Load imbalance is 4.77×: the slowest rank was busy far longer than the mean.
- **[ok]** 10 rank reattachment(s) occurred, up to incarnation 3: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

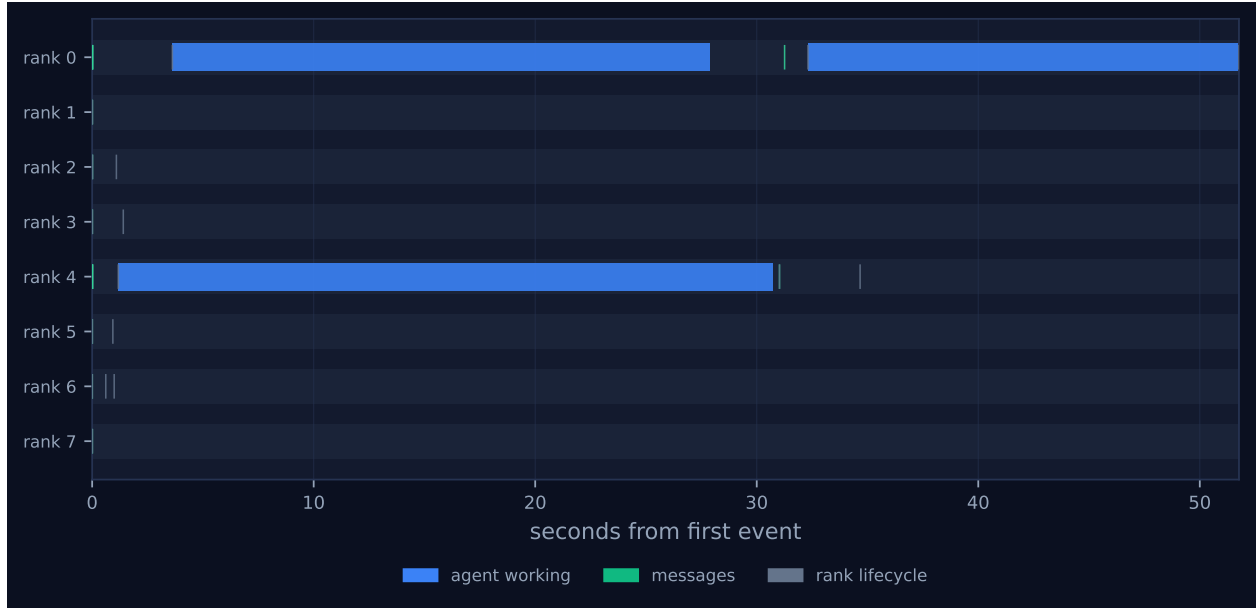


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

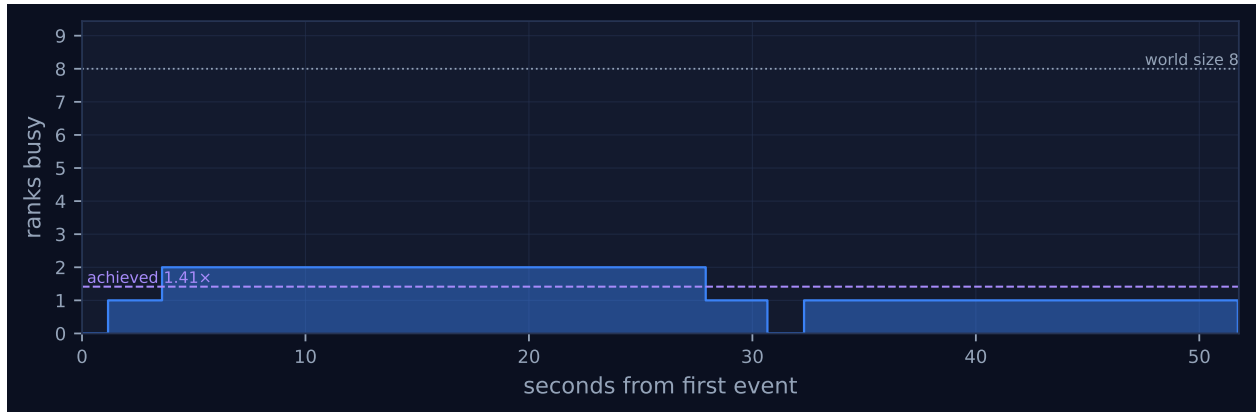


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

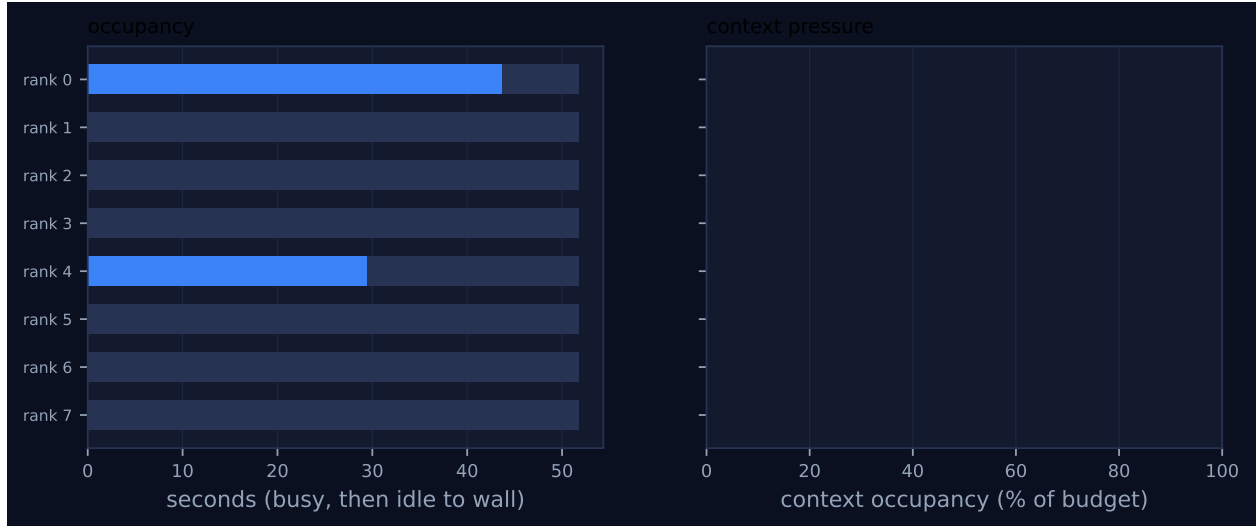


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	43.68	84.4	2	2	0	4	0	0.0	finalized
1	0.00	0.0	0	0	1	0	178	0.0	finalized
2	0.00	0.0	0	0	1	0	178	0.0	finalized
3	0.00	0.0	0	0	1	0	178	0.0	finalized
4	29.52	85.2	1	1	1	3	469	0.0	finalized
5	0.00	0.0	0	0	1	0	178	0.0	finalized
6	0.00	0.0	0	0	1	0	178	0.0	finalized
7	0.00	0.0	0	0	1	0	178	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	kary	–	8	8	2	2	7	7	1,397	51.76	51.76	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

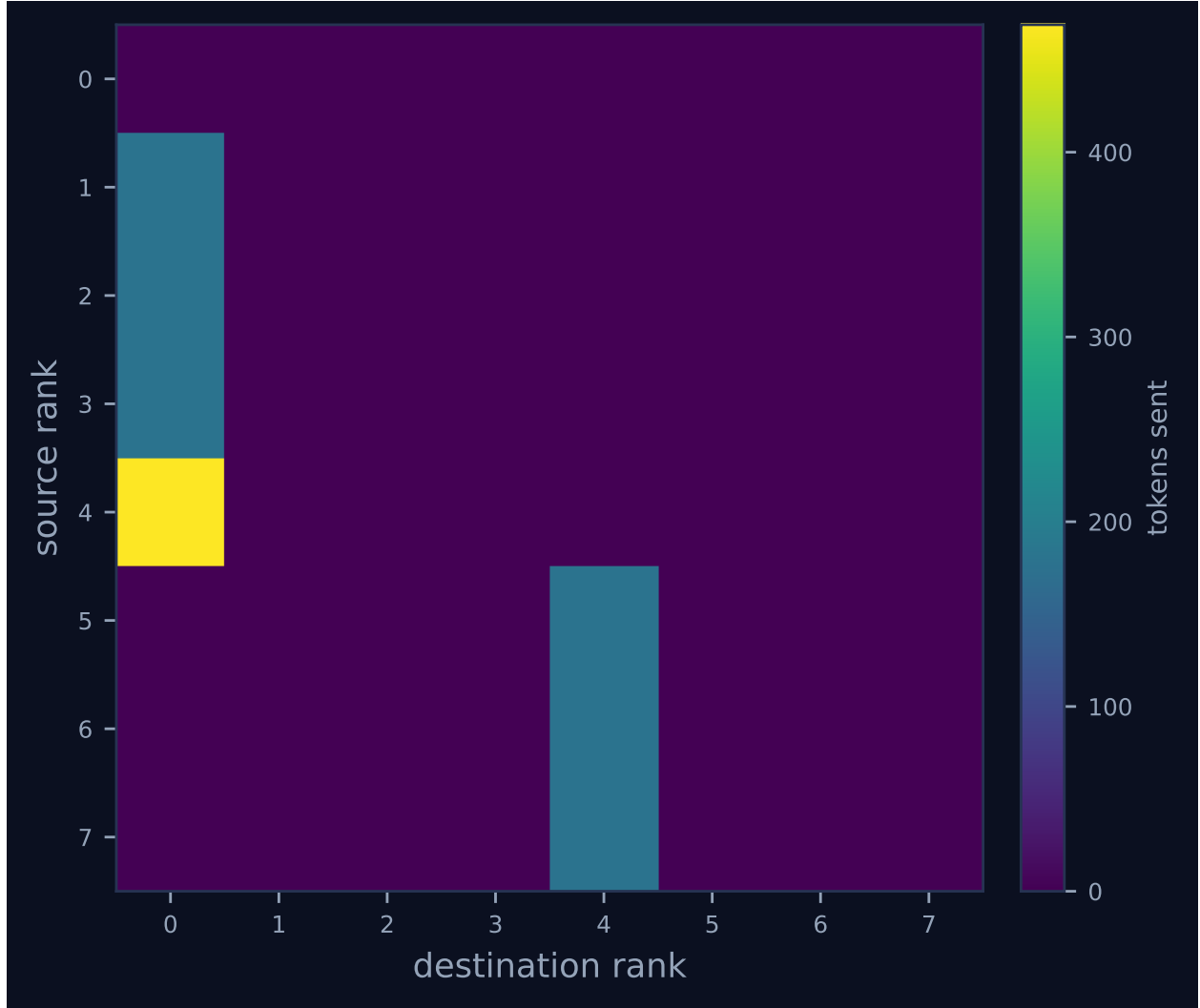


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

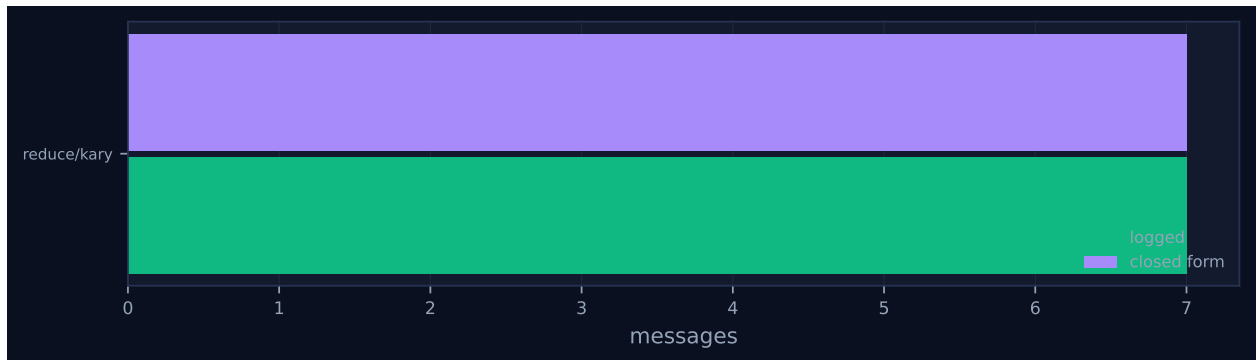


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

6 Reading the timeline

Figure 1 has the cleanest structure of any run in my set, and it is the k -ary tree drawn in two bands. Rank 0 and rank 4 hold bars together from about $t = 2$ s to $t = 30$ s; then rank 0 alone holds a second bar from about $t = 32$ s to 51.77 s. Ranks 1, 2 and 3 emit one tick each and exit; so do 5, 6 and 7. That is precisely what the algorithm’s two strides produce: at `stride = 1` the ranks with $vr \bmod 4 = 0$ — 0 and 4 — each collect three children and combine all four values in a single application, and at `stride = 4` rank 4 sends to rank 0, which combines two. Peak ranks busy is 2 of 8 and the serial fraction of active time is 50.3%, which is not inefficiency but the arithmetic of a two-level tree whose second level has one node. Sixty-three events, no trailing events at all, and the log’s last entry coincides with `job.finish` — the only broker-executed cell in the campaign with no post-completion tail, because the run finished before the worker pool started churning. The dashboard’s rank health table accordingly shows ranks 0 and 1 **finalized** rather than the wall of red **fail_stop** suspicions its siblings display.

Three bars, three latencies: 29.52 s for rank 4’s four-way fold, 24.27 s for rank 0’s, and 19.41 s for the final two-way fold, on prompts of 773, 773 and 1,014 tokens. Compare the binomial sibling, where the same 96 items required seven calls of 16–28 s. Per-call latency is essentially unchanged by widening the fan-in — 24.4 s mean here against 21.8 s there, on prompts $1.7\times$ larger — which is the empirical content of the claim that an agent is not single-ported. A rank that can absorb four reports in one prompt for the same 25 seconds it would have spent on two has no reason to prefer a binary tree, and the cost of the wider prompt shows up as 2,560 total prompt tokens against the binomial cell’s 4,220: fewer, larger prompts are cheaper in total than more, smaller ones, because the accumulator is not re-read at every level.

The communication matrix, Figure 4, is two columns: $1, 2, 3 \rightarrow 0$ and $5, 6, 7 \rightarrow 4$ at 178 tokens each, then $4 \rightarrow 0$ at 469, for seven messages and 1,537 tokens. That is the same message count as every other algorithm at this p — widening the tree changes rounds and applications, never messages, because every non-root rank sends exactly once — and it is 37% less traffic than the chain’s 2,448, since only one accumulator ever crosses the fabric instead of six.

7 Interpretation

The counts are by construction and all three closed forms are worth checking explicitly because this algorithm has no sibling in the collective sweeps and these cells are its only sweep-adjacent evidence. Fold depth: the `kary` branch advances `stride` by factors of k and increments depth once per level, giving $\lceil \log_4 8 \rceil = 2$, and the `coll.reduce` payload records exactly 2 against the binary tree’s 3 and the chain’s 7. Applications: each variadic call consumes k inputs and retires $k - 1$ of them, so a p -rank fold needs $\lceil (p - 1)/(k - 1) \rceil = 3$ at $p = 8$ with $k = 4$, and the trace holds exactly three `agent.call` events — two labelled `merge:k4:d1` and one `merge:k2:d2`, the label carrying the actual arity so the degenerate two-way top level is visible rather than inferred. Messages: seven, matching $p - 1$. The runtime also records `variadic: true` in the collective’s `extra` block, which matters because `Op.combine` silently degrades to a left fold when no variadic kernel exists, and a k -ary tree without one would spend exactly the depth it was built to save.

The result is that the quality argument for k -ary fails and the cost argument does not need it. Retention is 0.3854 here and 0.3854 in the depth-3 tree, the depth-7 chain and the depth-7 flat fold; I compared the survivor sets from the store and the symmetric difference is empty, and the final blob `9f300f3a45bf` is the same object the binomial and flat cells returned. The reason is the budget. An

incompressible item costs 11.81 tokens once quoted into the JSON array, so an enforced 450-token ceiling holds $450/11.81 = 38.1$ of them, and every merge in this campaign lands there: this run’s two level-0 folds returned 449 tokens with 38 identifiers and 437 with 37. Whichever application is last is the one that binds, so the surviving count is B/c regardless of how many applications preceded it, and there is no slack for a deeper tree’s earlier losses to compound into. Fold depth would govern fidelity only if interior nodes were allowed larger budgets than the root, which is what the weight-proportional variant in `semantic_op` exists to provide and what `-weighted` would have selected; every run in this campaign is `-f`, fixed budget, so that conditional remains untested.

Read as a cost decision, this cell dominates. Same 37 items, same seven messages, same zero hallucinated identifiers, and 3 applications instead of 7: 35% less wall time than the binomial tree, 47% fewer output tokens, 45% lower modelled cost, 39% fewer prompt tokens. Against the chain it is $12.3\times$ faster, though I would not defend that magnitude (see the threats). The one respect in which it is not free is context concentration: a level-0 rank here ingests 773 prompt tokens against 446 in the binary tree, so the binding constraint on k is the receiving rank’s context budget, exactly as `optimal_fanin` says. At the 32,768-token budget these runs declare that constraint is nowhere near binding — `optimal_fanin(32768, 450)` returns its cap of 32 — so $k = 4$ was a conservative choice made by the `-fanin` flag rather than by the policy, and the interesting configuration is the untested one: at $k \geq 8$ the whole reduction at $p = 8$ would be a single 8-way application at depth 1.

There is a defect to report, and it is in the cost model rather than the implementation. The collectives table shows rounds 2 against a prediction of 1, and yet the model column reads $\checkmark$. Two things are wrong. First, the tabulated entry ("`reduce`", "`kary`") in `cost.py` evaluates $\lceil \log_k p \rceil$ at `DEFAULT_FANIN = 8`, not at the fan-in the run used, so at $p = 8$ it predicts the single-application depth-1 tree while the run performed the $k = 4$ two-level one; the prediction is for a different experiment. `predict_kary(p, n, k)` takes an explicit k and exists precisely for this, but `analysis.py` consults `FORMULAS`. Second, the run-level check `Analysis.model_checks` counts only `messages_agree`, so `rounds_agree: false` is displayed in the table and in `metrics.json` but never reaches the finding that says “all 1 checkable collective(s) sent exactly the number of messages their cost expression predicts” — true as written, and reassuring in a way the underlying data does not support. The $p = 16$ `fid-synth` k -ary cell hides the same mismatch by coincidence, since $\lceil \log_8 16 \rceil$ and $\lceil \log_4 16 \rceil$ are both 2.

8 Threats to this reading

The executor is real: `executors` records `broker` $\times 8$ and `worker` $\times 10$, so the three merges were performed by models and the numbers are agent behaviour. The surrogate k -ary cells in `fid-synth` and `fid-inc-smoke` must not be pooled with this one, because the `function` executor’s operator drops whole accumulator blobs at a fixed probability per application and therefore implements a depth-loss model rather than testing it.

The dominance claim rests on one trial per algorithm, and the two comparisons are not equally safe. Against the binomial tree the mechanism and the measurement agree: 3 applications against 7, 51.77s against 80.27s, a ratio of 1.55 where the critical-path count predicts 1.5, and mean per-call latency differs by only 12% between the two runs. Against the chain the measurement is unreliable: the chain’s calls averaged 89.3s against this run’s 24.4s on comparable prompts, with 7.1s of broker queue wait in that run, so most of the $12.3\times$ is per-call latency inflation in the pool rather than structure. The structural prediction is $3.5\times$, and that is the figure I would defend.

Two accounting caveats specific to this cell. The final application is a two-way fold labelled `merge:k2:d2`, not a four-way one, because p is not a power of k ; so this run measures $k = 4$ at one level and $k = 2$ at the other, and a clean $k = 4$ measurement would need $p = 16$. The `fid-synth` k -ary cell has that geometry — four four-way folds and one four-way fold above them, five applications for $\lceil 15/3 \rceil$ — but it is a surrogate. Second, this run’s final result is byte-identical to the binomial and flat cells’ and the four cells shared one persistent pool of Cursor subagents. I ran the determinism test the contract requires and the executor fails it: two prompt digests are shared across cells in this campaign and both return different results. The identity is better explained by there being one canonical answer — the survivors are verbatim high-entropy strings emitted in rank order, so once the cut lands at 37 items the string is determined — but I cannot exclude pool memory, only observe that it would have nothing left to explain.

Finally, the token arithmetic uses the repository’s approximate counter (`tokenizer_exact: false`), which is the same counter the contract check uses, so the “38 items fit in 450 tokens” figure is internally consistent rather than externally valid; the \$0.0275 is modelled from \$3/Mtok in and \$15/Mtok out; and every `msg.recv` records `admitted: false` because the collective’s internal receives pass `admit=False`, so the 0% context occupancy in the per-rank table says nothing about whether a wider fan-in stresses a rank’s context. That is a real gap, since context is the quantity the k -ary argument says should bound k .